

halfedge: $\sqrt{3}$ 细分模块设计文档

src/subdiv/sqrt3.rs

halfedge 库

2026-07-04

目录

1	模块定位	2
1.1	核心 API	2
1.2	依赖关系	2
2	算法概述	2
3	面点 (Face Point)	2
4	边翻转 (Edge Flip)	2
4.1	内部边	2
4.2	边界边	3
4.3	翻转后三角形的方向推导	3
5	顶点松弛	3
5.1	内部顶点	3
5.2	边界顶点	3
6	规模变化	3
6.1	示例: icosphere(0)	4
7	算法流程	4
8	与 Loop / Catmull-Clark 对比	5
9	复杂度分析	5
10	退化健壮性	5
11	测试覆盖	6
12	设计取舍	6

1 模块定位

`sqrt3.rs` 在 `traversal.rs`、`io.rs` 之上叠加 **基于 $\sqrt{3}$ (Sqrt3) 细分的网格光滑能力**。参考 Kobbelt 2000 *A Variable Approximation Approach for Iterative Mesh Simplification*。

与 Loop 细分 ($4\times$ 面增长) 相比, $\sqrt{3}$ 细分每次面数增长 3 倍, 在渐进式细分中提供更细粒度的控制, 是 Loop 的重要补充。

1.1 核心 API

API	语义
<code>sqrt3_subdivide(&mesh)</code>	$\sqrt{3}$ 细分一次, 返回新网格, $F' = 3F$

1.2 依赖关系

依赖模块	用途
<code>storage</code>	<code>MeshStorage</code> 容器
<code>io</code>	<code>build_mesh_from_vertices_and_faces</code> (构建新网格)
<code>traversal</code>	<code>FaceHalfEdges</code> 、 <code>VertexRing</code> 、 <code>is_boundary_vertex</code>

2 算法概述

$\sqrt{3}$ 细分是**插值型**细分方案, 分三步:

1. **面点插入**: 每个三角面插入重心 $c_f = \frac{v_0+v_1+v_2}{3}$;
2. **边翻转**: 每条原始内部边 (v_0, v_1) 翻转为连接两侧面点 (c_1, c_2) ; 原始边界边保留 fan 三角形;
3. **顶点松弛**: 内部顶点按 α_n 权重松弛, 边界顶点保持原位置。

3 面点 (Face Point)

每个三角面 $f = (v_0, v_1, v_2)$ 的面点为重心:

$$c_f = \frac{v_0 + v_1 + v_2}{3}$$

与 Catmull-Clark (所有顶点平均) 不同, $\sqrt{3}$ 仅使用三角面的三个顶点。

4 边翻转 (Edge Flip)

4.1 内部边

设原始内部边 $h: v_0 \rightarrow v_1$, 所在面 f_1 (面点 c_1); twin: $v_1 \rightarrow v_0$, 所在面 f_2 (面点 c_2)。

fan 阶段两个三角形 (v_0, v_1, c_1) 和 (v_1, v_0, c_2) 被替换为翻转后的两个三角形:

$$(v_0, c_2, c_1), \quad (v_1, c_1, c_2)$$

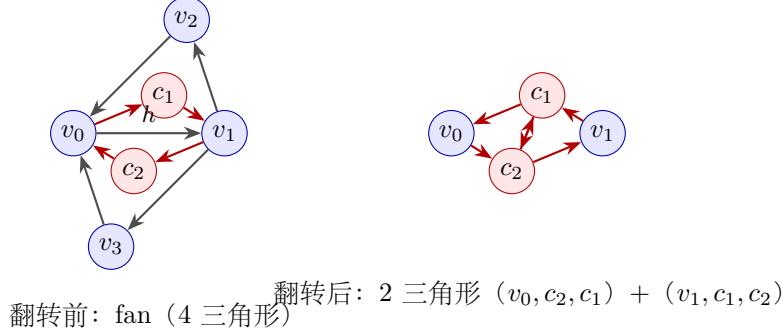
两者均为 CCW 朝向 (见下文方向推导)。

4.2 边界边

原始边界边 $h: v_0 \rightarrow v_1$, 所在面 f (面点 c); twin 无面。fan 三角形 (v_0, v_1, c) 保留 (无法翻转)。

4.3 翻转后三角形的方向推导

设 $h: v_0 \rightarrow v_1$, face $f_1 = (v_0, v_1, v_2)$ CCW (v_2 在边左侧), twin face $f_2 = (v_1, v_0, v_3)$ CCW (v_3 在边右侧)。 c_1 在 f_1 内 (左侧), c_2 在 f_2 内 (右侧)。



方向验证 (以 v_0 处三角形为例): 取 $v_0 = (0, 0)$, $v_1 = (1, 0)$, $v_2 = (0.5, 0.866)$, $v_3 = (0.5, -0.866)$, 则 $c_1 = (0.5, 0.289)$ (上), $c_2 = (0.5, -0.289)$ (下)。三角形 (v_0, c_2, c_1) 的叉积 $(c_2 - v_0) \times (c_1 - v_0) = 0.289 > 0$, 故 CCW。

5 顶点松弛

5.1 内部顶点

内部顶点 v (valence = n) 的新位置:

$$v' = (1 - \alpha_n) v + \frac{\alpha_n}{n} \sum_{j=1}^n v_j$$

其中 v_j 为原始 1-ring 邻居顶点 (旧位置), 权重:

$$\alpha_n = \frac{4 - 2 \cos(2\pi/n)}{9}$$

正则点 ($n = 6$): $\alpha_6 = \frac{4 - 2 \cos(\pi/3)}{9} = \frac{3}{9} = \frac{1}{3}$ 。

度数 3 ($n = 3$): $\alpha_3 = \frac{4 - 2 \cos(2\pi/3)}{9} = \frac{5}{9}$ 。

5.2 边界顶点

边界顶点保持原位置 (简化处理; 原始论文使用三次样条, 此处略)。

6 规模变化

设原始网格有 V 顶点、 E 边、 F 面:

量	公式	说明
V'	$V + F$	原始顶点 + 面点
F'	$3F$	内部边贡献 2 三角形 + 边界边贡献 1 三角形
E' (闭合)	$\frac{9F}{2}$	闭合三角网格 $3F' = 2E'$, 故 $E' = 9F/2$

闭合网格验证 (闭合三角网格): $V' = V + F$, $F' = 3F$, $E' = 3F'/2 = 9F/2$ 。

$$V' - E' + F' = (V + F) - \frac{9F}{2} + 3F = V + 4F - \frac{9F}{2} = V - \frac{F}{2}$$

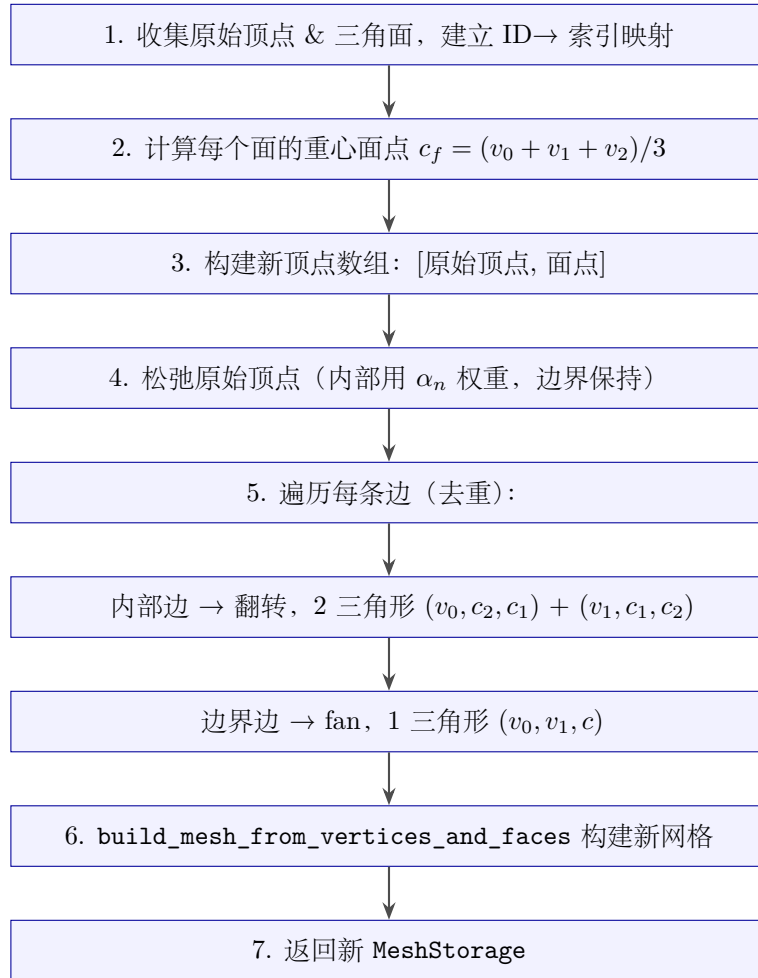
由 $V - 3F/2 + F = 2$ 得 $V - F/2 = 2$, 故 $V' - E' + F' = 2$ 。 Euler 示性数保持不变。

6.1 示例: icosphere(0)

$V = 12, F = 20, E = 30$ 。细分后:

$$V' = 12 + 20 = 32, \quad F' = 60, \quad E' = 90, \quad V' - E' + F' = 32 - 90 + 60 = 2.$$

7 算法流程



8 与 Loop / Catmull-Clark 对比

特性	Loop	Catmull-Clark	$\sqrt{3}$
面增长倍数	$4\times$	$4k \rightarrow 2k$ (三角化)	$3\times$
输入要求	三角形	任意多边形	三角形
顶点更新类型	逼近型	逼近型	插值型 (松弛后近似)
新顶点来源	边中点	边点 + 面点	面点 (重心)
边界处理	$1/8-3/4-1/8$	$1/8-6/8-1/8$	保持原位置
连续两次效果	光滑 ($4\times$)	光滑 ($4k\times$)	更细粒度控制 ($3\times$)

$\sqrt{3}$ 的 $3\times$ 面增长 (而非 $4\times$) 使其在渐进式细分中提供更细粒度的控制, 适合需要中间精度层级的场景。

9 复杂度分析

阶段	时间复杂度	空间复杂度
收集顶点 & 面	$O(V + F)$	$O(V + F)$
计算面点	$O(F)$	$O(F)$
顶点松弛	$O(V \cdot \bar{v})$	$O(V)$
边遍历 & 构建面	$O(E)$	$O(F')$
构建新网格	$O(V' + F')$	$O(V' + F')$
总计	$O(V + E + F)$	$O(V + F)$

其中 $\bar{v}$ 为平均顶点 valence (通常 6)。对闭合三角网格 $E = 3F/2$, 总计 $O(F)$ 。

10 退化健壮性

- 除零: α_n 在 $n = 0$ 时返回 0 (守卫);
- 孤立顶点: valence = 0 时保持原位置;
- 无效 ID: 所有 `mesh.get_*` 返回 None 时跳过;
- 空网格: $n_{\text{orig}} = 0$ 或 $n_{\text{faces}} = 0$ 时直接返回;
- 非三角面: 面边界环长度 $\neq 3$ 时跳过该面, 并通过 `eprintln!` 输出警告到 `stderr` (含跳过面数与替代建议 `catmull_clark_subdivide`);
- 拓扑断开: 半边无 twin 时跳过该边;
- 退化边: $i_o = i_t$ (自环) 时跳过。

11 测试覆盖

测试名	验证内容
规模验证	
<code>sqrt3_icosphere0_vertex_face_count</code>	<code>icosphere(0)</code> $V=12, F=20 \rightarrow V'=32, F'=60$
<code>sqrt3_icosphere1_vertex_face_count</code>	<code>icosphere(1)</code> $V=42, F=80 \rightarrow V'=122, F'=240$
拓扑校验	
<code>sqrt3_icosphere0_passes_validation</code>	<code>icosphere(0)</code> 细分后通过完整校验
<code>sqrt3_icosphere2_passes_validation</code>	<code>icosphere(2)</code> 细分后通过完整校验
<i>Euler</i> 示性数	
<code>sqrt3_preserves_euler_characteristic</code>	<code>icosphere(0..2)</code> 细分前后 $\chi = 2$
与 <i>Loop</i> 对比	
<code>sqrt3_vs_loop_face_growth</code>	<code>icosphere(1)</code> : <i>Loop</i> $F'=320$, $\sqrt{3}$ $F'=240$
连续细分	
<code>sqrt3_multiple_iterations_stay_valid</code>	连续 2 次 $\sqrt{3}$ 均通过校验, $V=92, F=180$
边界网格	
<code>sqrt3_single_triangle</code>	单三角形 $V'=4, F'=3$, 校验通过
<code>sqrt3_open_quad</code>	开四边形 $V'=6, F'=6$, 校验通过
位置正确性	
<code>sqrt3_face_point_at_centroid</code>	面点位于重心 $(1/3, 1/3, 0)$
<code>sqrt3_internal_vertex_relaxation</code>	四面体 v_0 松弛后位于 $(5/27, 5/27, 5/27)$
权重	
<code>sqrt3_alpha_regular_valence_6</code>	$\alpha_6 = 1/3$
<code>sqrt3_alpha_valence_3</code>	$\alpha_3 = 5/9$
<code>sqrt3_alpha_zero_valence_returns_zero</code>	$n = 0$ 守卫不 panic
空网格	
<code>sqrt3_empty_mesh</code>	空网格返回空网格
非三角面跳过 + 警告	
<code>sqrt3_skips_non_triangle_with_warning</code>	单四边形面输入: 跳过, 输出 0 面 (stderr 警告)

共 16 个单元测试, 全部通过。全库 `cargo test` 共 498 个用例, 0 失败。

12 设计取舍

- 为何用 $V' = V + F$ 而非 $V + E$ (**Loop**)? $\sqrt{3}$ 仅插入面点 (重心), 不插入边中点。面点通过边翻转自然连接, 无需额外的边点。这使面数增长 $3\times$ (而非 $4\times$)。
- 为何边界顶点保持原位置? 原始论文使用三次样条更新边界顶点, 但实现复杂。保持原位置是常见的简化策略, 确保边界不退化。

- **为何松弛用原始邻居而非新邻居（面点）？** 标准实现（Kobbelt 2000）使用原始 1-ring 邻居的旧位置计算松弛，保证算法的线性时间复杂度。若用新邻居（面点）需先构建新拓扑再回查。
- **为何直接构建新网格而非在原网格上操作？** 边翻转涉及大量拓扑修改（每条内部边都要翻转），在原网格上逐个翻转容易引入中间状态的不一致。直接收集新三角形列表一次性构建新网格更安全。
- **为何遍历所有半边而非仅 outgoing？** 每条无向边对应两条半边，用 `HashSet` 去重确保每条边只处理一次。遍历所有半边保证不遗漏任何边（包括边界边）。